

Martian Sieve

Our newly deployed facility on Mars has a fancy sieve used to categorize rocks. For any positive integer s the sieve can be configured so that rocks of size $< s$ fall through while rocks of size $\geq s$ remain in the sieve.

Our current issue with the sieve is that it got stuck in some specific setting s and stopped responding to commands we are sending from our HQ on Earth. We don't know the value s on which it is stuck and we need to determine it somehow.

Our engineers have realized that we already have a categorized collection of rock samples on Mars: for each i from 1 to n , inclusive, we have a rock of size i . It is guaranteed that $n \geq s$: the biggest rock in our collection will surely be caught by the sieve. It should be possible to use these rocks to determine s .

Task

You can perform arbitrarily many rounds. In each round:

- You pick an arbitrary subset of our categorized rocks.
- For each selected rock, sequentially: our robot will pick up the rock and place it into the sieve.
- Once all rocks are in the sieve, the robot will shake the sieve for a fixed amount of time.
- The robot will collect all rocks that fell through the sieve and **report their number back to you**. (In other words, you learn how many of the rocks you used fell through the sieve.)
- The robot will collect all rocks that remained in the sieve.
- Finally, the robot will return all the rocks used in the experiment back to their places in the collection.

Of course, when picking which rocks to use in a particular round, you can use all the information you obtained in previous rounds.

You are given the time c_r needed to process an individual rock and the time c_s that is needed for sieving and all other constant overhead during the process. Hence, a round in which we use r rocks will take $r \cdot c_r + c_s$ seconds.

Given n , c_r and c_s , calculate:

- The smallest x such that there is an algorithm that always correctly determines s and always takes at most x seconds to do so.
- One possible first round that leads to an optimal value of x .

Input format

The first line of each input file contains the number t of test cases. The specified number of test cases follows, one after another.

Each test case consists of a single line with three positive integers: n , c_r , and c_s .

Output format

For each test case output three lines:

- One line with the optimal worst-case time x .
- One line with a positive integer r : the number of rocks your optimal solution wants to use in the first round.
- One line with the sizes $s_1, \dots, s_r$ of those r rocks.

The numbers s_i must satisfy $1 \leq s_1 < s_2 < \dots < s_r \leq n$.

Subproblem M1 (12 points, public)

Input file: [M1.in \(/static/online2026/M1.caaa91024692cb6f1c09b935e92e2a24.in\)](/static/online2026/M1.caaa91024692cb6f1c09b935e92e2a24.in).

Constraints: $1 \leq t \leq 100$ and in each test $2 \leq n \leq 100$, $c_r = 1$, and $c_s = 100$.

Subproblem M2 (12 points, public)

Input file: [M2.in \(/static/online2026/M2.b9e73730497769fed684a9f77512b5a9.in\)](/static/online2026/M2.b9e73730497769fed684a9f77512b5a9.in).

Constraints: $1 \leq t \leq 100$ and in each test $2 \leq n \leq 100$, $c_r = 100$, and $c_s = 1$.

Subproblem M3 (7 points, public)

Input file: [M3.in \(/static/online2026/M3.83001845d85e850745d864450832cebe.in\)](/static/online2026/M3.83001845d85e850745d864450832cebe.in).

Constraints: $1 \leq t \leq 100$ and in each test $2 \leq n \leq 20$, $c_r \leq 100$, and $c_s \leq 100$.

Subproblem M4 (32 points, secret)

Input file: [M4.in \(/static/online2026/M4.407eae61cdda618cce79f9646de75f9e.in\)](/static/online2026/M4.407eae61cdda618cce79f9646de75f9e.in).

Constraints: $1 \leq t \leq 100$ and in each test $2 \leq n \leq 5000$, $c_r \leq 100$, and $c_s \leq 100$.

Subproblem M5 (37 points, secret)

Input file: [M5.in \(/static/online2026/M5.a47a0baa35a06135c79cbcd8eb743b39.in\)](/static/online2026/M5.a47a0baa35a06135c79cbcd8eb743b39.in).

Constraints: $1 \leq t \leq 100$ and in each test $2 \leq n \leq 10^{16}$, $c_r \leq 100$, and $c_s \leq 100$.

Example

input

```
3
2 100 100
4 4 30
4 25 3
```

output

```
200
1
1
42
3
1 2 3
56
1
2
```